Machine Learning Block1 Lab2-Group A4

Nazli Bilgic (nazbi056), Anastasia Piadi (anapi501), Max Björklund (maxbj107)

2023-12-06

Statement of Contribution

Anastasia (anapi501) was responsible for coding and writing analysis for assignment one.

Nazli (nazbi056) was responsible for coding and writing analysis for assignment two.

Max (maxbj107) was responsible for coding and writing analysis for assignment three.

Finalizing the last details of the report were done together.

Assignment 1. Explicit regularization

The tecator.csv contains the results of study aimed to investigate whether a near infrared absorbance spectrum can be used to predict the fat content of samples of meat. For each meat sample the data consists of a 100 channel spectrum of absorbance records and the levels of moisture (water), fat and protein. The absorbance is -log10 of the transmittance measured by the spectrometer. The moisture, fat and protein are determined by analytic chemistry. Divide data randomly into train and test (50/50) by using the codes from the lectures.

1) Assume that Fat can be modeled as a linear regression in which absorbance characteristics (Channels) are used as features. Report the underlying probabilistic model, fit the linear regression to the training data and estimate the training and test errors. Comment on the quality of fit and prediction and therefore on the quality of model.

```
## Loading required package: ggplot2

## Loading required package: lattice

## Loading required package: Matrix

## Loaded glmnet 4.1-8

## [1] "Training MSE: 0.00570911701403287"

## [1] "Test MSE: 722.429419254052"

##
## Call:
## lm(formula = Fat ~ . - Protein - Moisture, data = training_set)
##
```

```
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.201500 -0.041315 -0.001041  0.037636  0.187860
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.815e+01  5.488e+00  -3.306  0.01628 *
## Channel1     2.653e+04  1.126e+04   2.357  0.05649 .
## Channel2    -5.871e+04  3.493e+04  -1.681  0.14385
## Channel3     1.154e+05  7.373e+04   1.565  0.16852
## Channel4    -2.432e+05  1.175e+05  -2.070  0.08387 .
## Channel5     3.026e+05  1.193e+05   2.536  0.04430 *
## Channel6    -2.365e+05  8.160e+04  -2.898  0.02741 *
## Channel7     1.090e+05  3.169e+04   3.440  0.01380 *
## Channel8    -6.054e+04  1.508e+04  -4.015  0.00700 **
## Channel9     7.871e+04  2.160e+04   3.643  0.01079 *
## Channel10   -1.730e+04  1.640e+04  -1.055  0.33215
## Channel11    9.562e+04  3.529e+04   2.710  0.03512 *
## Channel12   -2.114e+05  6.198e+04  -3.410  0.01431 *
## Channel13    9.725e+04  4.424e+04   2.198  0.07026 .
## Channel14    5.296e+04  4.666e+04   1.135  0.29968
## Channel15   -7.855e+04  5.245e+04  -1.498  0.18491
## Channel16   -8.209e+03  1.893e+04  -0.434  0.67969
## Channel17    3.769e+04  1.987e+04   1.897  0.10666
## Channel18    3.306e+04  7.934e+03   4.167  0.00590 **
## Channel19   -8.405e+04  1.929e+04  -4.358  0.00478 **
## Channel20    1.510e+05  3.361e+04   4.492  0.00414 **
## Channel21   -2.069e+05  4.256e+04  -4.862  0.00282 **
## Channel22    1.348e+05  3.824e+04   3.526  0.01243 *
## Channel23   -4.094e+04  3.546e+04  -1.154  0.29222
## Channel24    2.023e+04  2.761e+04   0.733  0.49134
## Channel25    3.269e+03  1.071e+04   0.305  0.77045
## Channel26   -1.297e+04  7.636e+03  -1.699  0.14028
## Channel27    4.131e+03  1.422e+04   0.291  0.78120
## Channel28   -4.548e+03  2.988e+04  -0.152  0.88402
## Channel29    1.089e+04  1.768e+04   0.616  0.56072
## Channel30   -7.985e+04  2.653e+04  -3.010  0.02371 *
## Channel31    1.756e+05  5.279e+04   3.326  0.01589 *
## Channel32   -1.107e+05  2.904e+04  -3.813  0.00883 **
## Channel33   -6.525e+04  5.407e+04  -1.207  0.27294
## Channel34    1.007e+05  6.589e+04   1.528  0.17738
## Channel35   -2.841e+03  1.214e+04  -0.234  0.82266
## Channel36   -2.268e+04  2.295e+04  -0.988  0.36127
## Channel37   -4.479e+04  1.292e+04  -3.468  0.01334 *
## Channel38    3.209e+04  1.843e+04   1.742  0.13221
## Channel39    1.992e+04  2.067e+04   0.964  0.37246
## Channel40   -9.833e+03  2.431e+04  -0.404  0.69988
## Channel41    1.659e+04  3.648e+04   0.455  0.66531
## Channel42   -1.829e+04  3.528e+04  -0.519  0.62260
## Channel43   -2.423e+04  2.427e+04  -0.998  0.35669
## Channel44    3.246e+04  2.013e+04   1.613  0.15793
## Channel45   -8.089e+03  4.023e+04  -0.201  0.84728
## Channel46    7.065e+03  2.810e+04   0.251  0.80990
## Channel47   -4.062e+04  1.007e+04  -4.034  0.00685 **
```

```
## Channel48    9.080e+04  2.618e+04   3.469  0.01332 *
## Channel49   -6.647e+04  2.372e+04  -2.803  0.03105 *
## Channel50   -4.196e+04  2.856e+04  -1.469  0.19213
## Channel51    1.097e+05  5.572e+04   1.968  0.09661 .
## Channel52   -1.148e+05  6.376e+04  -1.800  0.12196
## Channel53    9.525e+04  7.450e+04   1.278  0.24830
## Channel54   -4.534e+04  7.363e+04  -0.616  0.56067
## Channel55   -1.535e+03  4.933e+04  -0.031  0.97618
## Channel56   -2.377e+03  2.109e+04  -0.113  0.91394
## Channel57    3.174e+04  1.005e+04   3.158  0.01961 *
## Channel58    2.221e+03  1.048e+04   0.212  0.83915
## Channel59   -8.504e+04  2.574e+04  -3.304  0.01634 *
## Channel60    6.382e+04  1.607e+04   3.972  0.00735 **
## Channel61    2.151e+04  1.234e+04   1.742  0.13211
## Channel62   -2.859e+04  1.065e+04  -2.685  0.03631 *
## Channel63    1.796e+04  9.187e+03   1.955  0.09838 .
## Channel64    5.759e+04  3.526e+04   1.633  0.15354
## Channel65   -1.470e+05  6.911e+04  -2.127  0.07752 .
## Channel66    9.121e+04  4.461e+04   2.045  0.08688 .
## Channel67   -5.733e+03  2.197e+04  -0.261  0.80288
## Channel68   -6.290e+04  2.192e+04  -2.870  0.02843 *
## Channel69    6.421e+04  2.074e+04   3.096  0.02121 *
## Channel70   -1.749e+04  1.581e+04  -1.106  0.31111
## Channel71   -7.248e+03  1.934e+04  -0.375  0.72075
## Channel72    3.406e+04  1.185e+04   2.873  0.02830 *
## Channel73   -2.100e+04  1.132e+04  -1.855  0.11308
## Channel74   -3.314e+04  1.220e+04  -2.717  0.03480 *
## Channel75    7.039e+04  2.054e+04   3.427  0.01402 *
## Channel76   -3.187e+04  1.736e+04  -1.836  0.11597
## Channel77    2.061e+04  1.810e+04   1.138  0.29832
## Channel78   -1.180e+04  2.273e+04  -0.519  0.62225
## Channel79    2.669e+04  2.997e+04   0.890  0.40750
## Channel80   -6.051e+04  1.483e+04  -4.080  0.00650 **
## Channel81    1.386e+03  2.628e+04   0.053  0.95966
## Channel82    1.020e+05  4.694e+04   2.173  0.07275 .
## Channel83   -1.706e+05  4.688e+04  -3.640  0.01083 *
## Channel84    1.097e+05  2.892e+04   3.792  0.00905 **
## Channel85   -1.294e+05  3.600e+04  -3.594  0.01145 *
## Channel86    2.130e+05  4.345e+04   4.903  0.00270 **
## Channel87   -1.198e+05  3.818e+04  -3.139  0.02011 *
## Channel88   -2.199e+04  6.085e+04  -0.361  0.73021
## Channel89    7.974e+04  5.077e+04   1.571  0.16733
## Channel90   -1.711e+05  5.499e+04  -3.112  0.02079 *
## Channel91    2.107e+05  6.406e+04   3.289  0.01663 *
## Channel92   -1.959e+05  7.171e+04  -2.733  0.03407 *
## Channel93    2.874e+05  9.937e+04   2.892  0.02762 *
## Channel94   -3.064e+05  9.601e+04  -3.191  0.01881 *
## Channel95    2.048e+05  6.220e+04   3.292  0.01656 *
## Channel96   -5.600e+04  2.929e+04  -1.912  0.10441
## Channel97   -1.318e+04  3.050e+04  -0.432  0.68065
## Channel98   -2.724e+04  2.107e+04  -1.292  0.24375
## Channel99    3.556e+04  1.382e+04   2.573  0.04218 *
## Channel100  -1.206e+04  4.264e+03  -2.828  0.03006 *
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.3191 on 6 degrees of freedom
## Multiple R-squared:      1,  Adjusted R-squared:  0.9994
## F-statistic:  1651 on 100 and 6 DF,  p-value: 1.058e-09
```

The prob. model for linear regression is:

$$y|x \ N(\theta^T x, \sigma^2)$$

$$y = \theta_0 + \theta + 1x_1 + ...\theta_p x_p + \epsilon$$

The y equals to Fat. The ϵ is the error term which stands for random errors found in the data that they are not captured by our model. The mean of the errors should be zero and independent of the x.

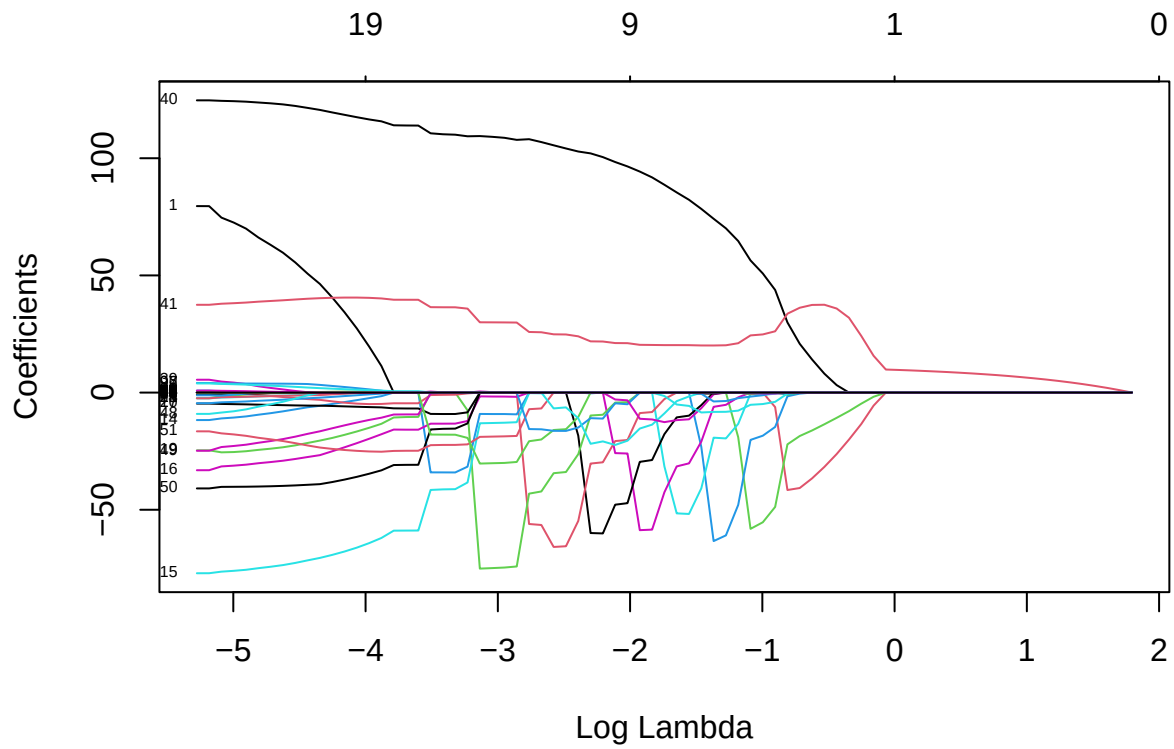
The large difference between the training and the test mean squared errors indicates overfitting. The model performs exceptionally well on training data but poorly in test data.

2)Assume now that Fat can be modeled as a LASSO regression in which all Channels are used as features. Report the cost function that should be optimized in this scenario.

$$\text{Lasso-cost function} = \sum_{i=1}^{n}(y_i - \sum_{j=1}^{p}\beta_j x_{ij})^2 + \lambda \sum_{j=1}^{p}|\beta_j|$$

The first term represents the residual sum of squares (RSS), which is a measure of the model's fit to the data. The second term is the LASSO penalty, which is the sum of the absolute values of the regression coefficients, scaled by the regularization parameter λ. This term eliminates some coefficients to zero. The regularization parameter λ controls the balance between fitting the model to the data and enforcing sparsity in the coefficients.

3)Fit the LASSO regression model to the training data. Present a plot illustrating how the regression coefficients depend on the log of penalty factor and interpret this plot. What value of the penalty factor can be chosen if we want to select a model with only three features?

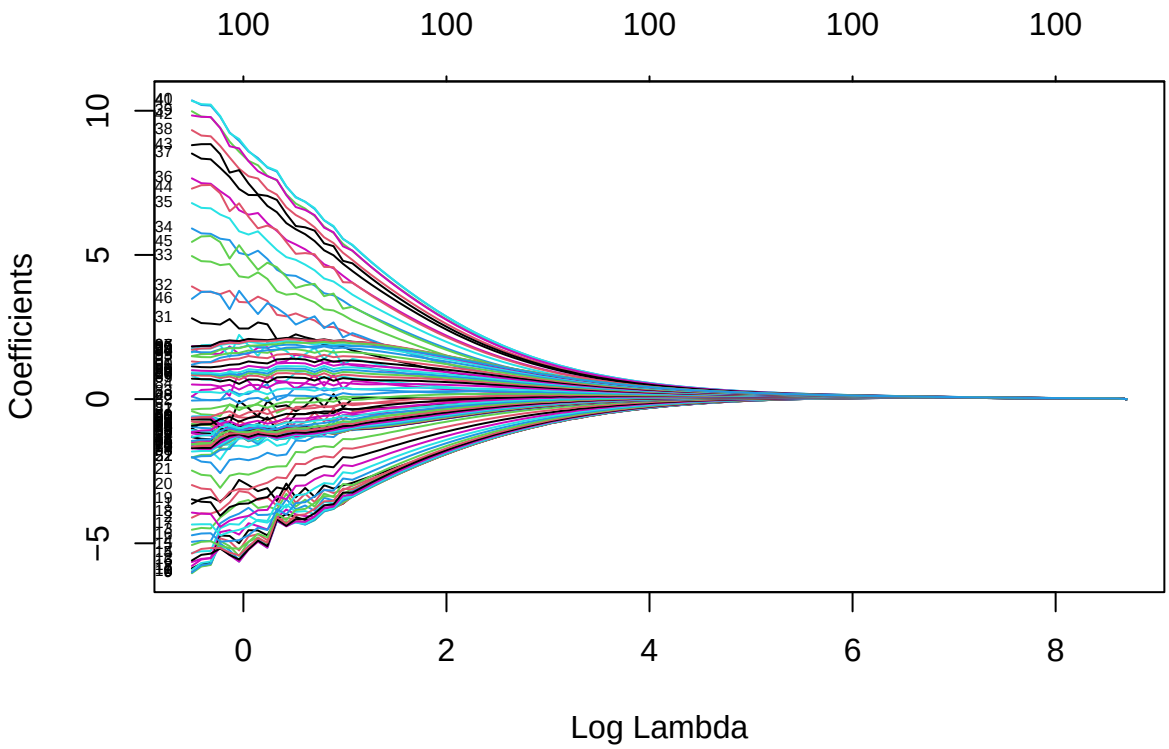



```
## lamda values for 3 features:  0.8530452 0.777263 0.7082131
```

By directrly observing the plot we can find the lamda that only the 3 lines are not equal to 0, which should be in a range and not distinct values.The model's output was 0.8530452 0.777263 0.7082131 which were the coefficients for only 3 distinct values.

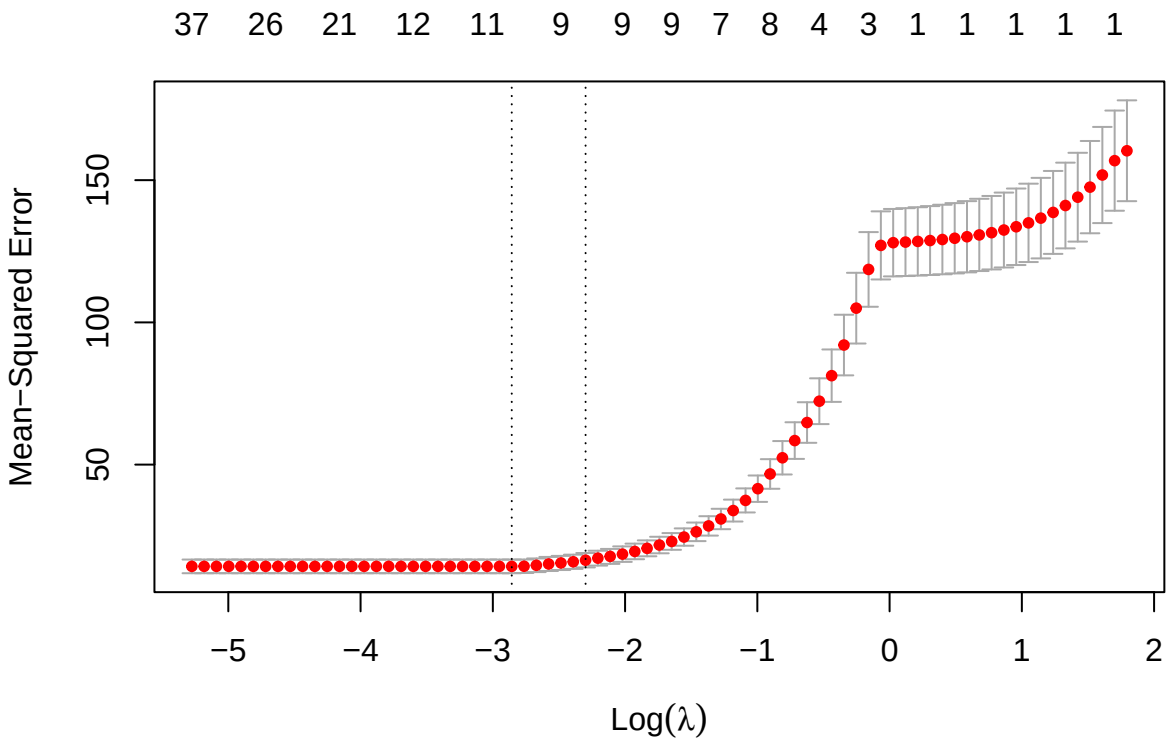
4)Repeat step 3 but fit Ridge instead of the LASSO regression and compare the plots from steps 3 and 4. Conclusions?

```
## lamda values for 3 features:  0.8530452 0.777263 0.7082131
```

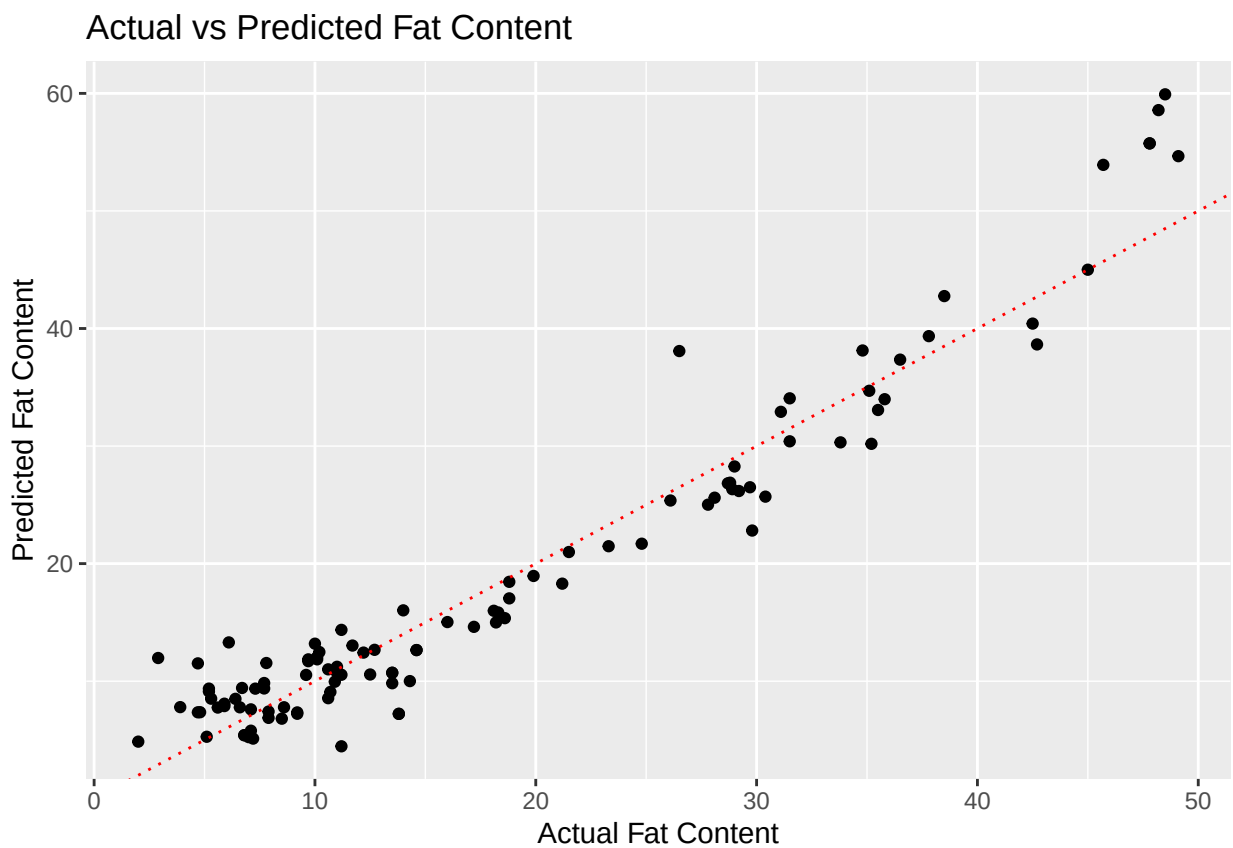


Lasso regression makes all the coefficients 0 , but ridge regression reduces them.

5)Use cross-validation with default number of folds to compute the optimal LASSO model. Present a plot showing the dependence of the CV score on log λ and comment how the CV score changes with log λ. Report the optimal λ and how many variables were chosen in this model. Does the information displayed in the plot suggests that the optimal λ value results in a statistically significantly better prediction than $\log(\lambda) = -4$? Finally, create a scatter plot of the original test versus predicted test values for the model corresponding to optimal lambda and comment whether the model predictions are good.




```
## Optimal lamda:  0.05744535

## Number of features in optimal model (including intercept):  8
```



In our Cross Validation plot the Mean Squared Error is highest at the lowest $\log\lambda$, indicating underfitting, decreases to a minimum suggesting optimal regularization, and then slowly rises with higher $\log\lambda$, indicating overfitting.

The optimal model was achieved when λ was equal to 0.057 and there were 8 features chosen.According to the plot the optimal λ does not result in a statistically significantly better prediction than $\log(\lambda) = -4$.

We can see from the plot that the real values are very close to predicted which shows that the model is efficient.However , there could be improvement at higher fat content levels.

Assignment 2. Decision trees and logistic regression for bank marketing

1) Import the data to R, remove variable "duration" and divide into training/validation/test as 40/30/30: use data partitioning code specified in Lecture 2a.

Data is divided according to lecture notes. Related code is in the Appendix.

2) Fit decision trees to the training data so that you change the default settings one by one (i.e. not simultaneously): a. Decision Tree with default settings. b. Decision Tree with smallest allowed node size equal to 7000. c. Decision trees minimum deviance to 0.0005. and report the misclassification rates for the training and validation data. Which model is the best one among these three? Report how changing the deviance and node size affected the size of the trees and explain why.

When we change the minimum allowed node size of the tree, the tree become simpler with less splits. The tree with node size 7000 has fewer nodes.

When the minimum node size is large tree creates larger nodes with more data and with this we get a simpler tree.

Setting the minimum deviance to 0.0005 increased the number of nodes. Lower deviance caused a more complex decision tree. Lowering the deviance level, model start to pay more attention to the training data(noise and minor fluctuations), as a result we get more complex tree.

Missclassification rates;

From our misclassificaion error results we can see that the default setting tree and the tree with minimum node number has the same misscalassification error values for both validation and train data.

The tree with deviance 0.0005 has the lowest training error, it fits the training data better the other two models. The validation error is higher for this model wehn we compare it to the other two models we have.

To conclude, the default setting tree and the tree with minimum node number has better performance because they have lower validation error. The model which has a better performance on validation data set is prefered. Because, this data is not used on training so its a new data set for the model. But the misclassification values that we get for these models doesnt have very big difference so other factors should be considered for a better decision.

```
## missclassificaiton error train data, default tree: 0.1048441

## missclassificaiton error validation data, default tree: 0.1092679

## missclassificaiton error train data, tree min node 7000: 0.1048441

## missclassificaiton error validation data, tree min node 7000: 0.1092679

## missclassificaiton error train data, tree deviance: 0.09400575

## missclassificaiton error validation data, tree deviance: 0.1119221
```

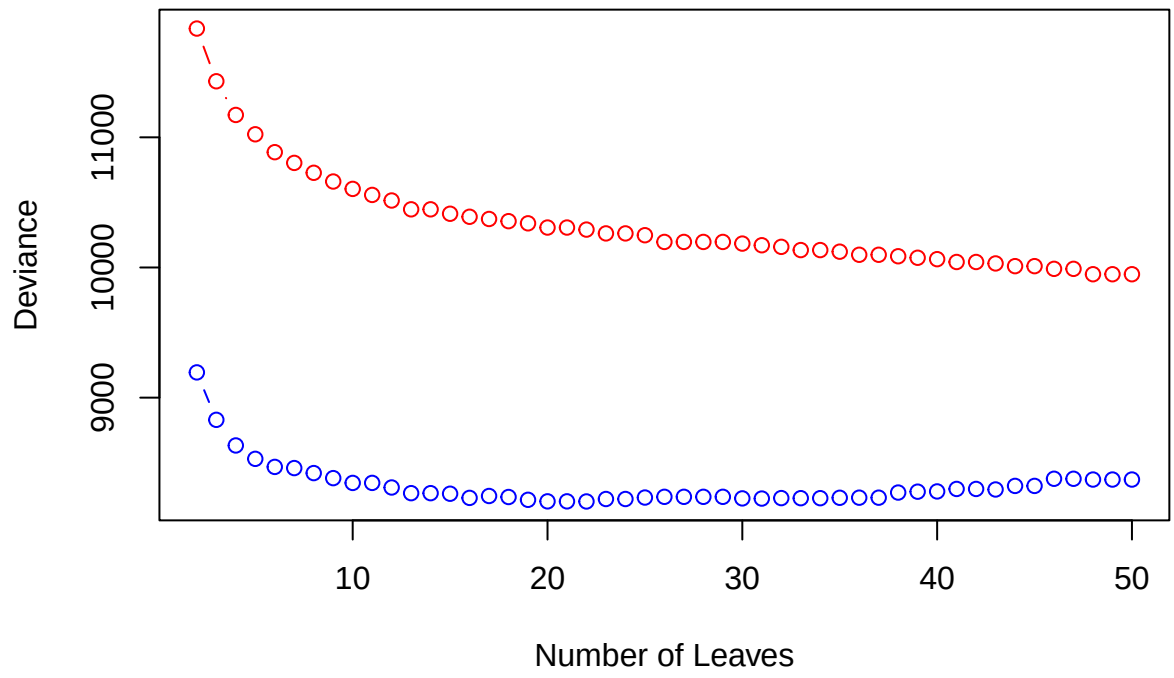
3) Use training and validation sets to choose the optimal tree depth in the model 2c: study the trees up to 50 leaves. Present a graph of the dependence of deviances for the training and the validation data on the number of leaves and interpret this graph in terms of bias-variance tradeoff. Report the optimal amount of leaves and which variables seem to be most important for decision making in this tree. Interpret the information provided by the tree structure (not everything but most important findings).

At the beginning of the graph where there are few leaves, the deviance is high for both the validation and training sets. This can be because of model is underfitting; model is too simple and has high bias. From the plot we can see that as the number of leaves increases, the deviance for both validation and training sets decrease. This indicates that adding more leaves(more complex starts to get things right more often) reduce bias. With the range of leaves that we plot the model doesn't seem to have high variance problem. Because the error doesn't increase on the validation data.

At teh optimal point which we calculated poitn where the tree has 22 leaves. This point the validation data the model achieves a good balance between bias and variance.

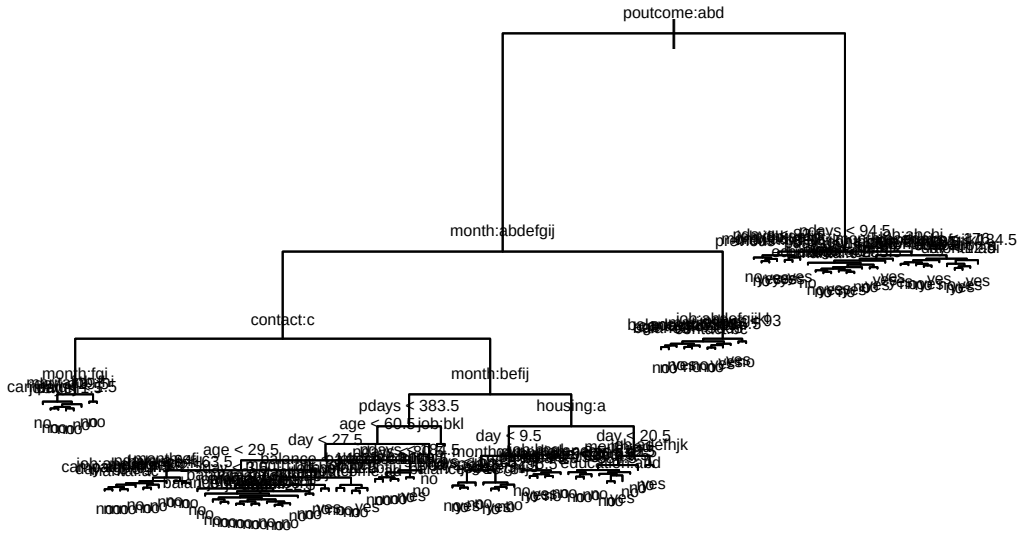
From the tree we can see that, poutcome is the root node(first split) is the most important variable. This variable does the best job at separating the data into classes at the start. We can also consider the variables which are closer to the top one like; month, contact. The ones that are closer to the top has more affect on the model than the ones that are down. Month variable is for subsequent splits, its also important for the model. contact variable used in he lower splits its less important than 'poutcome' or 'month'. pdays, age,

campaign, marital, job, balance, housing these variables are also used in the splits that we have showing they also have affect but much more lower.




```
## optimal number of leaves: 22
```

```
## deviance at optimal number-leaves: 8202.509
```



4) Estimate the confusion matrix, accuracy and F1 score for the test data by using the optimal model from step 3. Comment whether the model has a good predictive power and which of the measures (accuracy or F1-score) should be preferred here.

F1 score that we found is approximately 0.285 which is low. Low F1 score can show that model is not as effective at correctly predict the positive('yes') class. From the confusion matrix we can also see that the model is better at predicting the 'no' but it struggles with the 'yes'-positive class. Accuracy that we calculated is high. This can be happening because of the dataset(imbalanced dataset situation) that we have.('no' instances being higher than the 'yes' instances in dataset) F1 is a better measurement choice for this model. Because, it shows how many correct positive predictions were made and how many actual positives were caught. Since predicting 'yes' is failing for the model F1 score can be prioritized. F1 score balances the precision and recall and is generally more informative for imbalanced datasets.

```
## F1 Score: 0.2848752
```

```
## Accuracy: 0.8922884
```

```
##          Actual
## Predicted    no   yes
##       no  11812  1294
##       yes   167   291
```

5) Perform a decision tree classification of the test data with the following loss matrix, and report the confusion matrix for the test data. Compare the results with the results from step 4 and discuss how the rates has changed and why.

When we compare the confusion matrix results from step 4 and 5, the matrix in step 4 has more true negatives which means it was better at correctly identifying 'no' cases. The matrix in step 5 has a higher number of true positives, this was better at correctly predicting 'yes' when the actual outcome was 'yes'.

Step 4 has fewer false positives(less likely to predict incorrectly 'yes' when actual value was 'no'). Step 5 has less false negatives the probability of predicting 'no' when the real answer was 'yes'.

According to the loss matrix that is given in the question we added a threshold value we are checking if probability of being 'no' divided by probability of being 'yes' value is greater than 5 or not and if it is greater then 5 we say 'no'. Otherwise we assign 'yes'. This control value(5) is from the loss matrix. by adding this we are saying 'no' if we are really sure about it(under a certain condition). Because of this model predicts more yes values and 'yes' prediction values got higher in the second confusion matrix.

```
##          Actual
## Predicted    no   yes
##       no  11030   771
##       yes   949   814
```

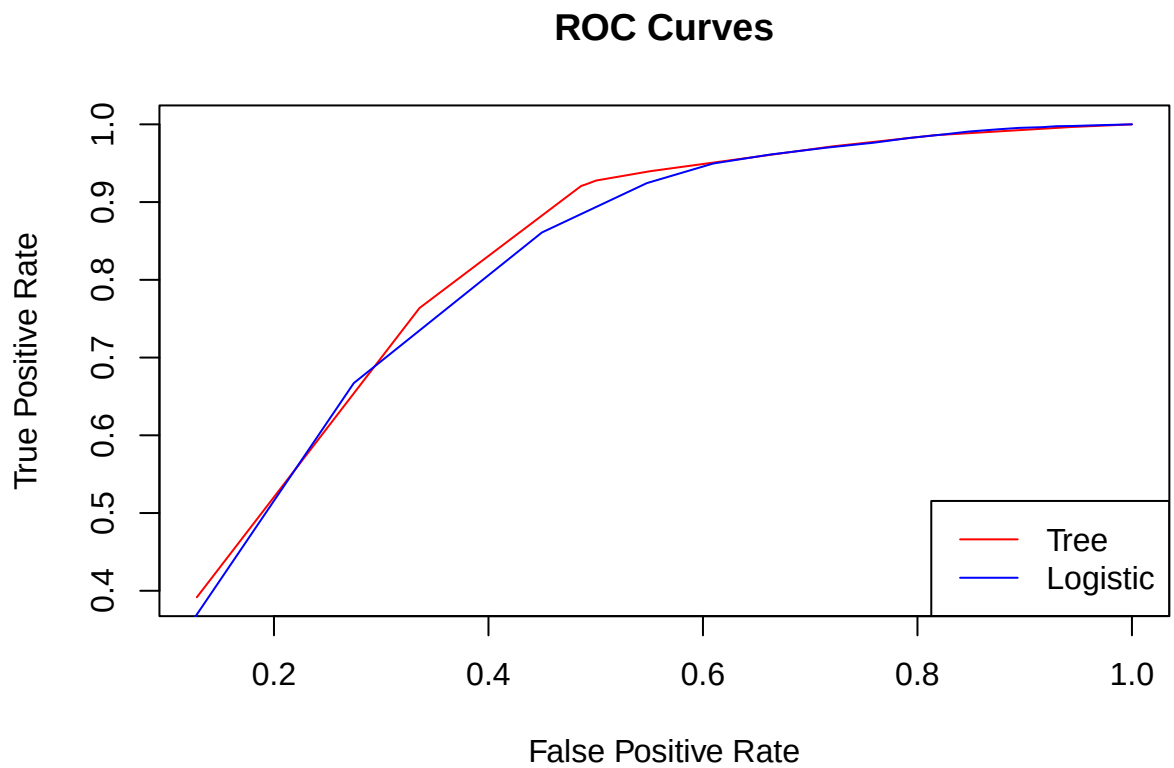
6) Use the optimal tree and a logistic regression model to classify the test data by using the following principle: Y^=yes if p(Y='yes'|X)> pi, otherwise Y^=no where pi = 0.05,0.1,0.15, ... 0.9,0.95. Compute the TPR and FPR values for the two models and plot the corresponding ROC curves. Conclusion? Why precision- recall curve could be a better option here?

ROC curves for two models are very close to each other. The curves are close to the top-left corner, both models have a good true positive rate compared to the false positive rate, which indicates a good ability to identify positive cases at the same time keeping the false positives low.

TPR(True positive rates) for both models increases with the FPR(False positive rates).

Precision recall curve can be better to use at this case. Because, when the classes are imbalanced the ROC curve can give optimistic results. When we print the proportion of 'yes' and 'no', we can see that our classes are also imbalanced(no:0.88, yes:0.12). Precision recall curves are more focused on how good the model is at spotting the less common class in the dataset. So, when we don't have many 'yes' cases, looking at precision recall curves can be a better way to see if your model is good at finding those 'yes' cases.

sources; https://towardsdatascience.com/precision-recall-curve-is-more-informative-than-roc-in-imbalanced-data-4c95250242f6, https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4349800/, https://en.wikipedia.org/wiki/Precision_and_recall




```
## [1] "proportion of the minority class to the total-imbalance:"

##
##        no       yes
## 0.8831466 0.1168534
```

Assignment 3. Principal components and implicit regularization

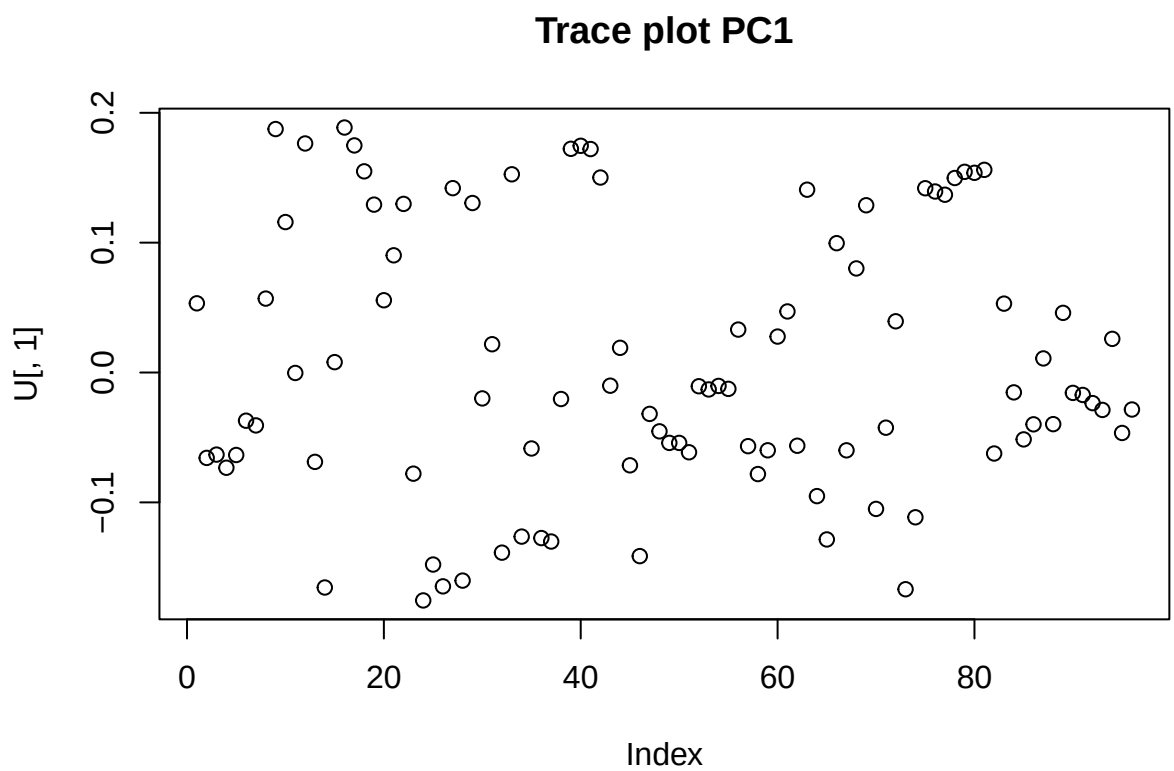
3.1

Scale all variables except of ViolentCrimesPerPop and implement PCA by using function eigen(). Report how many components are needed to obtain at least 95% of variance in the data. What is the proportion of variation explained by each of the first two principal components?

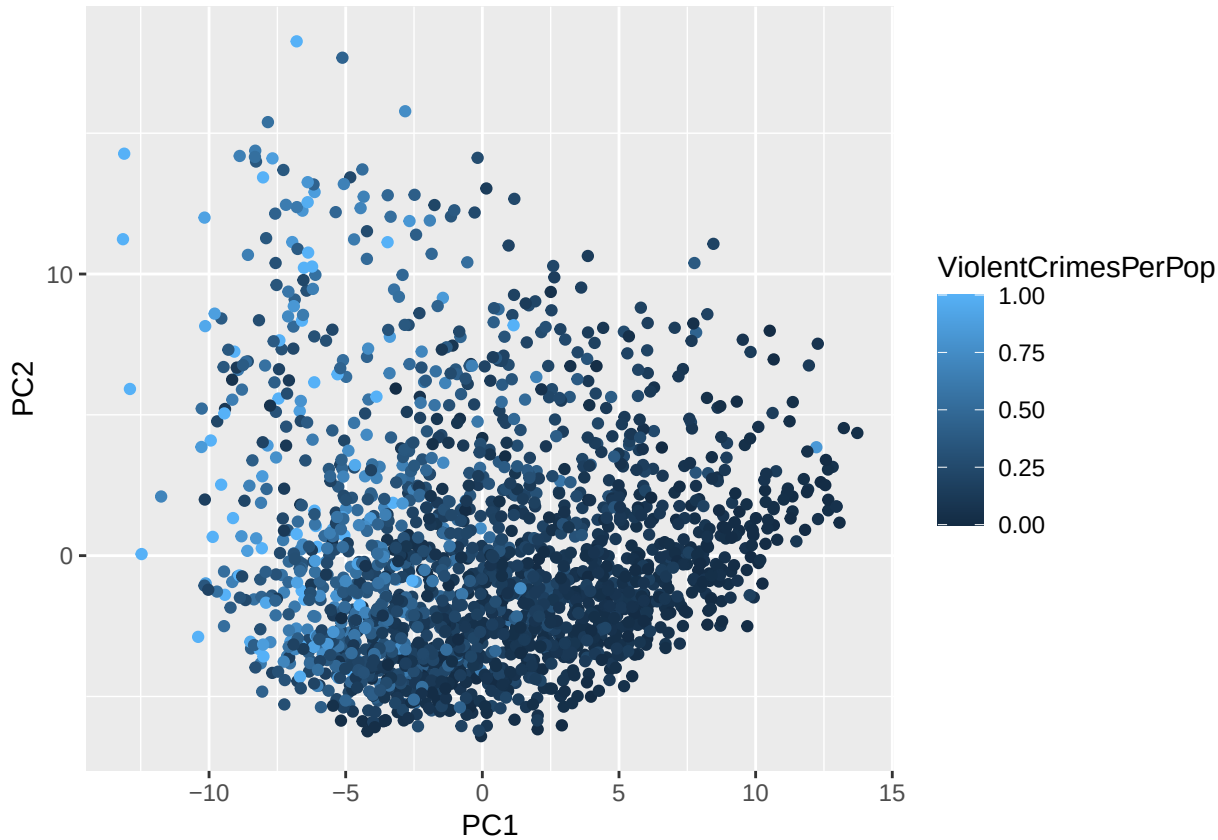
```
## [1] "How many components before 95% of variance is explained?"

## [1] 34

## [1] "Proportion of variation explaiend by PC1"

## [1] 0.256369

## [1] "Proportion of variation explaiend by PC2"

## [1] 0.1706683
```

3.2

Repeat PCA analysis by using princomp() function and make the trace plot of the first principle component. Do many features have a notable contribution to this component? Report which 5 features contribute mostly (by the absolute value) to the first principle component. Comment whether these features have anything in common and whether they may have a logical relationship to the crime level. Also provide a plot of the PC scores in the coordinates (PC1, PC2) in which the color of the points is given by ViolentCrimesPerPop. Analyse this plot (hint: use ggplot2 package).




```
## [1] "5 features that contribute the most to PC1"

##      medFamInc      medIncome     pctWInvInc PctPopUnderPov      perCapInc
##      0.1886516      0.1874868      0.1763855      0.1755022      0.1748879
```



Do many features have a notable contribution to this component?

Yes, the trace plot shows that there are multiple features that are well below- or well above 0 contribution.

Report which 5 features contribute mostly

medFamInc, medIncome, pctWInvInc, PctKids2Par, PctPopUnderPov

The first two represents median household income for the given region, where the first is for family households and the second is for overall household income. Also the third also regards household economics as it represents the percentage of households in the region with investment and rent income. This is logically tied to crime level as higher income regions usually have lower reported crime rates.

The contrast to this is in the fifth feature which represents percentage of people in the region who lives in poverty. With the same logic state above, this could affect crime rate.

The fourth is percentage of kids in family housing with two parents. It is known that regions with single households has in general more criminality.

Also provide a plot of the PC scores in the coordinates (PC1, PC2) in which the color of the points is given by ViolentCrimesPerPop.

Higher crime rate seem to correspond with lower PC1-score. PC2 is a little bit more difficult to interpret, but seem to have a positive effect, as none of the lowest scoring observations in PC2 have high crime rate.

3.3

Split the original data into training and test (50/50) and scale both features and response appropriately, and estimate a linear regression model from training data in which ViolentCrimesPerPop is target and all other

data columns are features. Compute training and test errors for these data and comment on the quality of model.

```
## [1] "Training Error (MSE)"
```

```
## [1] 0.06786482
```

```
## [1] "Test Error (MSE)"
```

```
## [1] 0.08874766
```

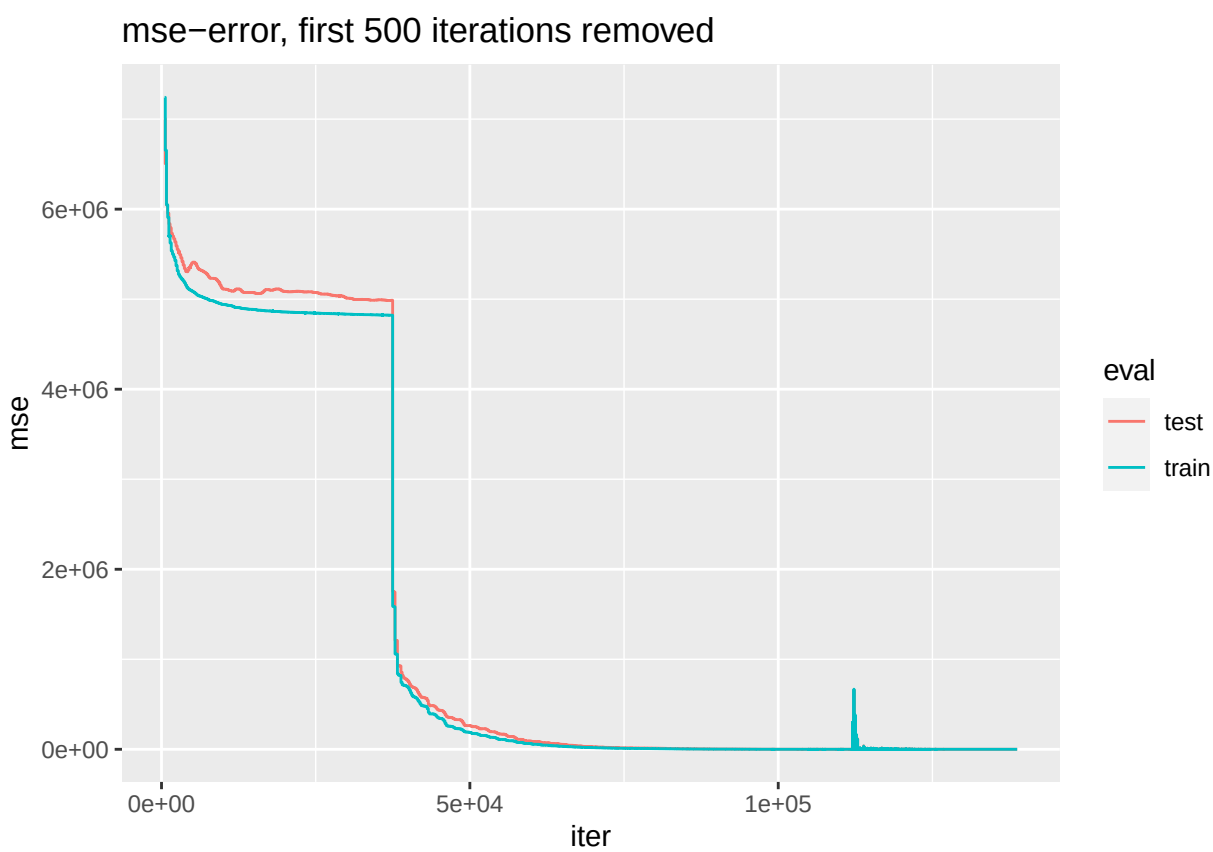
Comment on the quality of model

As the range of our response variable is between 0-1, a mean squared error of between 0.06 and 0.09 is not good. This means that the average error on every observation fit are about $\sqrt{0.09} = 0.3$.

3.4

Implement a function that depends on parameter vector theta and represents the cost function for linear regression without intercept on the training data set. Afterwards, use BFGS method (optim() function without gradient specified) to optimize this cost with starting point 0 and compute training and test errors for every iteration number. Present a plot showing dependence of both errors on the iteration number and comment which iteration number is optimal according to the early stopping criterion. Compute the training and test error in the optimal model, compare them with results in step 3 and make conclusions.

a. Hint 1: don't store parameters from each iteration (otherwise it will take a lot of memory), instead compute and store test errors directly. b. Hint 2: discard some amount of initial iterations, like 500, in your plot to make the dependences visible.



```
## [1] "Iteration with lowest training-error"
```

```
## [1] 138742
```

```
## [1] "Iteration with lowest test-error"
```

```
## [1] 134843
```

```
## [1] "Number of iterations"
```

```
## [1] 138742
```

Do do early stopping, we would further need to split the data into three parts to include a validation set as well. As these instructions limits us to just a training and a test set, it would be more responsible applying the early stopping-criterion on the training error, since we are not supposed to optimize our model after test-data. But, if the objective is to avoid overfitting, we have to pick something else than training data.

As expected, the last iteration gave the lowest training-error. This is because the optim()-function will most likely stop optimizing when cost function is minimized.

Test-error was lowest at an earlier iteration. To avoid overfitting, we will chose this iteration.

```
## [1] "Training Error (MSE) 3.3"
```

```
## [1] 0.06786482
```

```
## [1] "Test Error (MSE) 3.3"
```

```
## [1] 0.08874766
```

```
## [1] "Training Error (MSE) 3.4"
```

```
## [1] 5.959513e-05
```

```
## [1] "Test Error (MSE) 3.4"
```

```
## [1] 6.739738e-05
```

As per usual are the training errors lower than their corresponding test error. Although the model has gotten much better with the cost function optimization, lowering the MSE-error with many decimals. Optimizing this has taken a lot of time and computing power compared to the lm()-function, which most likely gets its poor estimates from shortcuts and generous convergence limits.

Appendix

Chunk Label: Question 1

```r
library(caret)
library(glmnet)
library(tree)
```

Chunk Label: Question 1.1

```r
tecator_data <- read.csv("tecator.csv")
tecator_data <- tecator_data[-1] # Remove the sample column
set.seed(12345)

#split data into training and testing sets
data_size <- nrow(tecator_data)
sample_ids <- sample(1:data_size, floor(data_size * 0.5))
training_set <- tecator_data[sample_ids, ]
testing_set <- tecator_data[-sample_ids, ]

#linear regression model excluding Protein and Moisture
linear_model <- lm(Fat ~ . - Protein - Moisture, data = training_set)

#make predictions and calculate Mean Squared Error (MSE) for training and testing sets
train_predictions <- predict(linear_model, training_set)
train_mse <- mean((train_predictions - training_set$Fat) ^ 2)

test_predictions <- predict(linear_model, testing_set)
test_mse <- mean((test_predictions - testing_set$Fat) ^ 2)
print(paste("Training MSE:", train_mse))
print(paste("Test MSE:", test_mse))
summary(linear_model)
```

Chunk Label: Question 1.3

```r
covariate_matrix <- training_set[, !names(training_set) %in%
                                  c("Fat", "Protein", "Moisture")]
response_variable <- training_set$Fat

#LASSO model
lasso_model <- glmnet(as.matrix(covariate_matrix), response_variable,
                      alpha = 1, family="gaussian")

#plot showing the dependence of coefficients
plot(lasso_model, xvar="lambda", label=TRUE)

#lamda values (three non-zero coefficients)
selected_lambdas <- which(lasso_model$df == 3)
cat("lamda values for 3 features: ", lasso_model$lambda[selected_lambdas], "\n")
```

Chunk Label: Question 1.4

```
#lamda values (exactly three non-zero coefficients)
selected_lambdas <- which(lasso_model$df == 3)
cat("lamda values for 3 features: ", lasso_model$lambda[selected_lambdas], "\n")

#ridge Regression
ridge_model <- glmnet(as.matrix(covariate_matrix), response_variable, alpha = 0, family="gaussian")

#plot for Ridge Regression
plot(ridge_model, xvar="lambda", label=TRUE)
```

Chunk Label: Question 1.5

```
#cross-validation for optimal LASSO model
cv_lasso_model <- cv.glmnet(as.matrix(covariate_matrix), response_variable,
                            alpha=1, family="gaussian")
plot(cv_lasso_model)

#optimal lamda from cross-validation
optimal_lambda <- cv_lasso_model$lambda.min
cat("Optimal lamda: ", optimal_lambda, "\n")

#prediction using the optimal model
optimal_pred <- predict(cv_lasso_model, s = optimal_lambda,
                        newx = as.matrix(testing_set[, 1:100]))
optimal_test_mse <- mean((optimal_pred - testing_set$Fat)^2)

#number of coefficients in the optimal model
num_coef <- cv_lasso_model$nzero[which(cv_lasso_model$lambda.min == cv_lasso_model$lambda)]
cat("Number of features in optimal model (including intercept): ", num_coef, "\n")

#plot of actual vs predicted Fat content
ggplot(data = data.frame(Actual = testing_set$Fat, Predicted = optimal_pred),
  aes(x = Actual, y = optimal_pred)) +
  geom_point() +
  geom_abline(intercept = 0, slope = 1, linetype = "dotted", color = "red") +
  labs(x = "Actual Fat Content", y = "Predicted Fat Content",
       title = "Actual vs Predicted Fat Content")
```

Chunk Label: Question 2.1

```
  library(dplyr)
  library(tree)
  library(rpart.plot)
  library(ggplot2)
  library(randomForest)
  library(caret)
```

```r
library(pROC)

bank_data <- read.csv("bank-full.csv", sep = ";",
                      stringsAsFactors = TRUE) #factor the non numeric columns

data<- select(bank_data, -"duration")

n=dim(data)[1]
set.seed(12345)
id=sample(1:n, floor(n*0.4))
train=data[id,]
id1=setdiff(1:n, id)
set.seed(12345)
id2=sample(id1, floor(n*0.3))
valid=data[id2,]
id3=setdiff(id1,id2)
test=data[id3,]
```

Chunk Label: Question 2.2.a

```r
n=dim(train)[1]
model_tree <- tree(y ~ ., data = train)

#plot(model_tree)
#text(model_tree, pretty=0)
#title("Decision Tree Plot1")
#model_tree
#summary(model_tree)
```

Chunk Label: Question 2.2.b

```r
tree_model_7000 <- tree(y ~ ., data = train, control =
                          tree.control(nobs = nrow(train), minsize = 7000))

#print(tree_model_7000)
#plot(tree_model_7000)
#text(tree_model_7000)
#title("Decision Tree Plot min node size 7000")
```

Chunk Label: Question 2.2.c

```r
tree_model_deviance <- tree(y ~ ., data = train, control =
                              tree.control(nobs = nrow(train), mindev = 0.0005))
```

Chunk Label: Question 2.2 missclassification errors

```
pred_train_model1<-predict(model_tree,train,type="class")
pred_valid_model1<-predict(model_tree,valid,type="class")

missclassEr_train_model1<-mean(pred_train_model1 != train$y)
missclassEr_valid_model1<- mean(pred_valid_model1 != valid$y)

cat("missclassificaiton error train data, default tree:",
    missclassEr_train_model1,"\n" )
cat("missclassificaiton error validation data, default tree:",
    missclassEr_valid_model1,"\n" )


pred_train_model2<-predict(tree_model_7000,train,type="class")
pred_valid_model2<-predict(tree_model_7000, valid, type="class")

missclassEr_train_model2<-mean(pred_train_model2 != train$y)
missclassEr_valid_model2<-mean(pred_valid_model2 != valid$y)

cat("missclassificaiton error train data, tree min node 7000:",
    missclassEr_train_model2,"\n" )
cat("missclassificaiton error validation data, tree min node 7000:",
    missclassEr_valid_model2,"\n" )

pred_train_model3<-predict(tree_model_deviance,train,type="class")
pred_valid_model3<-predict(tree_model_deviance, valid, type="class")

missclassEr_train_model3<-mean(pred_train_model3 != train$y)
missclassEr_valid_model3<-mean(pred_valid_model3 != valid$y)

cat("missclassificaiton error train data, tree deviance:",
    missclassEr_train_model3,"\n")
cat("missclassificaiton error validation data, tree deviance:",
    missclassEr_valid_model3,"\n" )
```

Chunk Label: Question 2.3

```
trainScore = rep(0, 50)
validScore = rep(0, 50)

for (i in 2:50) {
  prunedTree = prune.tree(tree_model_deviance, best = i)
  pred = predict(prunedTree, newdata = valid, type = "tree")
  trainScore[i] = deviance(prunedTree)
  validScore[i] = deviance(pred)
}

#print(testScore)
```

```r
plot(2:50, trainScore[2:50], type = "b", col = "red",
     ylim = c(min(trainScore[2:50], validScore[2:50]),
              max(trainScore[2:50], validScore[2:50])),
      xlab = "Number of Leaves",
     ylab = "Deviance"
     )

points(2:50, validScore[2:50], type = "b", col = "blue")

optimal_leaves <- which.min(validScore[2:50])
optimal_deviance <- min(validScore[2:50]) #min deviation
# 1 added because the indexing starts at 2
cat("optimal number of leaves:", optimal_leaves + 1, "\n")
cat("deviance at optimal number-leaves:", optimal_deviance, "\n")

plot(tree_model_deviance)
text(tree_model_deviance, cex=0.5)  # Adjust cex to change text size for tree

#tree_model_deviance #print the tree
```

Chunk Label: Question 2.4

```r
optimal_tree <- prune.tree(tree_model_deviance, best = optimal_leaves) #tree in optimal size
predicted_test <- predict(optimal_tree, newdata = test, type = "class")
confusion_matrix_test <- table(Predicted=predicted_test, Actual=test$y)

#true positive
TP <- confusion_matrix_test[2, 2]  # assume 'yes' is coded as 2
#true negative
TN <- confusion_matrix_test[1, 1]  # assume 'no' is coded as 1
FP <- confusion_matrix_test[1, 2]  # false positives
FN <- confusion_matrix_test[2, 1]  # False negatives

# F1 score
precision <- ifelse((TP + FP) > 0, TP / (TP + FP), 0)
recall <- ifelse((TP + FN) > 0, TP / (TP + FN), 0)
f1_score_test <- ifelse((precision + recall) > 0, 2 *
                          (precision * recall) / (precision + recall), 0)

#accuracy
accuracy_test <- (TP + TN) / sum(confusion_matrix_test)

cat("F1 Score:", f1_score_test, "\n")
cat("Accuracy:", accuracy_test, "\n")
print(confusion_matrix_test)
```

Chunk Label: Question 2.5

```r
#use vector to get probabilities, use the optimal tree
```

```r
predicted_test <- predict(optimal_tree, newdata = test, type = "vector")

ratio <- predicted_test[, 'no'] / predicted_test[, 'yes']

#if prob no divided by prob yes bigger than 5 it's 'no'
#otherwise 'yes'
final_prediction <- ifelse(ratio > 5, "no", "yes")

confusion_matrix_2 <- table(Predicted = final_prediction, Actual = test$y)
print(confusion_matrix_2)
```

Chunk Label: Question 2.6

```r
logistic_model <- glm(y ~ ., data = test, family = "binomial")


# predict probabilities on the test data
prob_tree <- predict(optimal_tree, newdata = test, type = "vector")[, "yes"]
prob_logistic <- predict(logistic_model, newdata = test, type = "response")

# initialize vectors to store TPR and FPR for each of the model
thresholds <- seq(0.05, 0.95, by = 0.05)
tpr_tree <- fpr_tree <- tpr_log <- fpr_log <- numeric(length(thresholds))

for (i in seq_along(thresholds)) {
  pi <- thresholds[i]

  # calssify based on threshold
  pred_tree <- ifelse(prob_tree > pi, "yes", "no")
  pred_logistic <- ifelse(prob_logistic > pi, "yes", "no")

  #  predictions to factor and explicitly set the levels to match test$y
  pred_tree <- factor(pred_tree, levels = levels(test$y))
  pred_logistic <- factor(pred_logistic, levels = levels(test$y))


  # confusion matrices
  cm_tree <- confusionMatrix(as.factor(pred_tree), as.factor(test$y))
  cm_log <- confusionMatrix(as.factor(pred_logistic), as.factor(test$y))

  # TPR and FPR
  tpr_tree[i] <- cm_tree$byClass["Sensitivity"]
  fpr_tree[i] <- 1 - cm_tree$byClass["Specificity"]

  tpr_log[i] <- cm_log$byClass["Sensitivity"]
  fpr_log[i] <- 1 - cm_log$byClass["Specificity"]
}

#roc curves
plot(fpr_tree, tpr_tree, type = "l", col = "red", xlab = "False Positive Rate",
     ylab = "True Positive Rate", main = "ROC Curves")
```

```r
lines(fpr_log, tpr_log, type = "l", col = "blue")
legend("bottomright", legend = c("Tree", "Logistic"), col = c("red", "blue"),
       lty = 1)

#proportion of the minority class to the total
print("proportion of the minority class to the total-imbalance:")
prop.table(table(test$y))
```

Chunk Label: 3_0

```r
knitr::opts_chunk$set(echo = TRUE)
data <- read.csv("communities.csv")
```

Chunk Label: 3_1

```r
#Removing non-predictive variables
data <- data[,-c(1:5)]
data[,-96] <- scale(data[,-96])

pca <- function(data){
  X <- as.matrix(data)
  n <- nrow(X)
  S <- (1/n)*t(X) %*% X
  eigens <- eigen(S)

  Z <- X %*% eigens$vectors

  cumvar <- cumsum(eigens$values)/sum(eigens$values)

  return(list(eigenvalues = eigens$values, cumvar = cumvar, pca = Z))
}

pcares <- pca(data)

print("How many components before 95% of variance is explained?")
sum(pcares$cumvar < 0.95)+1
print("Proportion of variation explaiend by PC1")
pcares$cumvar[1]
print("Proportion of variation explaiend by PC2")
pcares$cumvar[2]-pcares$cumvar[1]
```

Chunk Label: 3_2

```r
pcares2 <- princomp(data)
U <- pcares2$loadings

plot(U[,1], main = "Trace plot PC1")
```

```r
print("5 features that contribute the most to PC1")
head(sort(abs(U[,1]), decreasing = T), n = 5)

library(ggplot2)

visdata <- data.frame(PC1 = pcares2$scores[,1], PC2 = pcares2$scores[,2], ViolentCrimesPerPop = data$Vi

ggplot(data = visdata, aes(x = PC1, y = PC2, color = ViolentCrimesPerPop)) + geom_point()
```

Chunk Label: 3_3

```r
set.seed(12345)

#Indices from 50%
trainindex <- sample(1:nrow(data), size = 0.5*nrow(data))

train <- data[trainindex,]
test <- data[-trainindex,]

mod <- lm(ViolentCrimesPerPop ~ . -1, data = train)

mse <- function(Y, Y_hat){

  mean((Y-Y_hat)^2)

}
print("Training Error (MSE)")
mse(train$ViolentCrimesPerPop, mod$fitted.values)
print("Test Error (MSE)")
mse(test$ViolentCrimesPerPop, predict(mod, newdata = test))
```

Chunk Label: 3_4_1

```r
set.seed(12345)

# Starting vector
theta0 <- rep(0, times = 96)

# Emtpy vectors to save train and test errors
trainss <- c()
testss <- c()

# Cost function, inspired by the minus-log-likelihood function from last lab
cost <- function(theta){

  # Extracting sigma, adding a small number to keep it from being 0
  sigma <- theta[length(theta)]+1e-5
  # Keeping the rest of the vector
  theta <- theta[-length(theta)]
```

```r
  prediction_train <- as.matrix(train[,!(colnames(data) %in% "ViolentCrimesPerPop")]) %*% theta
  y <- train$ViolentCrimesPerPop
  n <- length(y)

  # Sum of squared error
  sum_square <- sum((y - prediction_train)^2)

  # Minus log_likelihood as our optim-function will minimize
  minus_log_likelihood <- -(-n/2 * log(2 * pi * sigma^2) - 1/(2 * sigma^2)
                            * sum_square)

  # Saving sum of squared-error to vector
  trainss[length(trainss)+1] <<- sum_square


  #Repeat for test
  prediction_test <- as.matrix(test[,!(colnames(data) %in%
                                          "ViolentCrimesPerPop")]) %*% theta
  y <- test$ViolentCrimesPerPop
  n <- length(y)

  sum_square <- sum((y - prediction_test)^2)

  testss[length(testss)+1] <<- sum_square

  return(minus_log_likelihood)




}

opt <- optim(par = theta0, fn = cost, method = "BFGS", control = list(maxit = 5000))

plotdata <- rbind(data.frame(iter = 1:length(trainss),mse = trainss/nrow(train), eval = "train"),
                  data.frame(iter = 1:length(testss), mse = testss/nrow(test), eval = "test"))

ggplot(data = plotdata[!(plotdata$iter %in% 1:500),], aes(x = iter, y = mse, color = eval)) + geom_line
  ggtitle("mse-error, first 500 iterations removed")

print("Iteration with lowest training-error")
which.min(trainss)
print("Iteration with lowest test-error")
which.min(testss)
print("Number of iterations")
length(trainss)
```

Chunk Label: 3_4_2

```r
print("Training Error (MSE) 3.3")
mse(train$ViolentCrimesPerPop, mod$fitted.values)
```

```r
print("Test Error (MSE) 3.3")
mse(test$ViolentCrimesPerPop, predict(mod, newdata = test))
print("Training Error (MSE) 3.4")
sqrt(trainss[which.min(testss)])/length(trainss)
print("Test Error (MSE) 3.4")
sqrt(testss[which.min(testss)])/length(testss)
```